Zero-Cost Sovereign Computing: A Reproducible Multi-Node Cluster from mDNS, a Mesh VPN, Cloud Tunnels, and Free-Tier VPSs*

Vibhav Aggarwal[a,*]

[a]*Independent Researcher, Gurugram, 122001, Haryana, India*

Abstract

Institutional computing resources are unevenly distributed, and commercial cloud costs create a structural barrier for independent researchers, small manufacturers, and individual practitioners who nonetheless require capable compute for AI workloads, knowledge-graph systems, and persistent services. We present a **reference cluster** of eleven heterogeneous nodes whose measured recurring subscription fees to cloud providers, SaaS vendors, and network services total approximately Rs 300/month (≈USD 3.60/month), composed of recycled consumer hardware, enterprise servers acquired secondhand, a consumer WiFi router upgraded to OpenWrt and treated as a first-class mesh node, an ARM64 smartphone converted into a Linux server, two Oracle Always-Free-Tier virtual machines, and a small number of cheaply-bought new single-board computers. We explicitly do not account for electricity, hardware amortization, or operator labor, none of which have been instrumented; Section 4 makes the scope of the cost claim precise. Within each LAN segment nodes find one another by zero-configuration **mDNS/Avahi** (`.local`); across segments they are stitched together by a self-hosted **Headscale** (v0.28) control plane; selected services are published to the public Internet via **Cloudflare Tunnels**; and the cluster is exposed to agents as a federation of seven **Model Context Protocol (MCP)** servers. The architecture is deliberately minimal: no Kubernetes control plane, no commercial observability stack, no paid managed services. We describe the topology, authentication model, service mesh, and practical gotchas encountered over ~20 months of continuous operation (Feb 2025 – Oct 2026), and argue that this design is a viable, reproducible pattern that any independent researcher or small organization can build for themselves — without institutional affiliation, grant funding, or venture capital. We release a complete reference configuration.

Keywords: self-sovereign infrastructure, mesh VPN, mDNS, Headscale, Cloudflare Tunnel, free-tier VPS, edge computing, MCP federation, heterogeneous clusters, home lab

1. Introduction

The dominant narrative in modern compute assumes one of two regimes: (1) hyperscale commercial cloud, where any workload can be rented for a monthly fee, or (2) institutional compute, allocated through university or corporate membership. For individuals and small organizations operating outside both — independent researchers, family-run manufacturers, solo practitioners bridging traditional knowledge with computational methods — neither option is straightforwardly accessible. Cloud bills scale faster than personal savings. Institutional access presumes institutional affiliation.

Meanwhile, the hardware substrate required for substantive work is increasingly available at near-zero marginal cost. Consumer laptops from 2015 retain usable CPUs and GPUs. IBM and HP enterprise servers from 2008-2012 appear on secondhand markets at prices below a single month of equivalent cloud rental. Android smartphones can be turned into portable ARM64 Linux servers, whether flashed with a mainline Linux distribution [1] or running a chrooted Linux userland on a custom Android ROM. Cloudflare's free tier includes unmetered tunnels [2]; Oracle Cloud Infrastructure (OCI) offers two AMD-based VMs in perpetuity at no charge [3]; Headscale [4] provides a self-hostable, open-source reimplementation of Tailscale's coordination server [5].

The question this paper addresses is practical: **can heterogeneous recycled hardware, spanning consumer, enterprise, mobile, and cloud tiers, be unified into a single operational fabric at effectively zero recurring cost, while remaining suitable for non-trivial workloads?**

We answer affirmatively, by construction, and document the resulting system.

1.1. Contributions

1. **A legacy-first hardware-acquisition strategy paired with a cheapest-viable-add-on corollary**, with a documented inventory (Table 3) and an explicit two-column cost accounting that distinguishes *historical acquisition cost* (≥ USD 13,000 across the hardware's prior owners and prior uses) from *marginal cost to this cluster* (≈ USD 625). Six of the eleven nodes were already in the operator's inventory from prior personal or professional uses.

*Preprint. Also archived at Zenodo, doi:10.5281/zenodo.19654425 (19 April 2026 (v1); v2 2026-10-14), CC BY 4.0. Corresponding author: Vibhav Aggarwal, `vibhav@vibhavaggarwal.com`.

*Corresponding author. ORCID: 0009-0000-7686-7119.

Email address: `vibhav@vibhavaggarwal.com` (Vibhav Aggarwal)

The GPU node in particular illustrates the strongest form of the principle: a 12-GPU mining rig that originally cost Rs 5,50,000 in 2020, whose capital was paid back through mining revenue before Ethereum's PoS migration, and that now contributes to the cluster at zero marginal capital. Worked examples of the cheapest-add-on corollary include a **Google Coral USB Accelerator** (~Rs 12,000, deployed on office-server) in place of a discrete NVIDIA GPU (~Rs 70,000–80,000) for edge ML inference, and a **Raspberry Pi 5** (~Rs 13,225) in place of a full x86 node for the home-server role.

2. **A reference topology** combining Headscale, Cloudflare Tunnels, and OCI Always-Free, sufficient to serve as a starting point for others building comparable systems.
3. **A cost accounting** demonstrating that recurring subscription fees to cloud and SaaS providers can be driven to roughly Rs 300/month or less, and a clear statement of what is excluded from that figure (electricity, hardware amortization, operator labor).
4. **A service-federation pattern** using MCP across seven filesystem-scoped endpoints, enabling agentic LLMs to span the full cluster.
5. **Practical gotchas** observed over ~20 months of continuous operation, including Headscale-vs-Tailscale-SSH port conflicts, Docker bridge IP ban-listing, and the impossibility of raw TCP over certain Headscale DERP paths.
6. **A position argument** that this design is not merely hobbyist but a viable sovereignty-and-sustainability-first infrastructure pattern for independent research and small-organization compute — one that reduces rather than generates e-waste.

1.2. Non-goals

We do not claim this system matches hyperscaler reliability, throughput, or feature surface. We do not propose it as a replacement for production SaaS in regulated or high-availability contexts. The evaluation is operational (does it work, at what cost?), not benchmark-driven.

2. Related Work

Home labs and self-hosting. The r/selfhosted and r/homelab communities have produced extensive practical literature on self-hosted service stacks, typically focused on media streaming, file sync [6], and home automation [7]. Academic treatment is comparatively thin; most experience reports appear in practitioner venues such as USENIX ;login: [8] rather than peer-reviewed conferences. This paper is intended as a contribution to that practitioner literature.

Mesh VPNs. WireGuard [9] is the underlying cryptographic tunnel technology we rely on. Tailscale [5] popularized WireGuard-over-NAT mesh networking with a managed central coordination server. Headscale [4] is the open-source reimplementation of that coordination server; we use it here. ZeroTier [10] provides an alternative layer-2 fabric; Nebula [11] emphasizes certificate-based trust. We compare these in §3.3.

Cloudflare Tunnels and NAT-traversal for inbound access. Cloudflare's Tunnel product [2] enables inbound connectivity without port-forwarding, and integrates with Cloudflare Access for zero-trust policies. Alternatives include ngrok [12], frp [13], and Tailscale Funnel [14].

Heterogeneous compute. The HPC community has long worked on heterogeneous schedulers (Slurm, Kubernetes); we operate at a much smaller scale where no scheduler is needed — workloads are assigned by role (see §3.5).

Model Context Protocol (MCP). Introduced by Anthropic in late 2024 [15] and formalized in an open specification [16], MCP defines a JSON-RPC-based interface between LLM clients and external tool/resource servers. MCP servers are typically deployed as sidecars to applications; our contribution is deploying them as persistent per-node filesystem servers over HTTPS, using `mcp-proxy` [17], and thereby enabling an agentic LLM to traverse an entire cluster of nodes.

Mobile-as-server. A smartphone can host persistent Linux services in one of two ways: by flashing a mainline distribution such as postmarketOS [1] — which runs upstream Linux kernels on mainlined Qualcomm platforms, the OnePlus 6 (SDM845) being among the best-supported devices — or by running a Linux userland in a chroot atop a stock/custom Android ROM. We use the latter (a Kali chroot on LineageOS): it keeps the phone's Android cellular stack intact, which is what lets the node serve as a cellular SMS/voice gateway, while still exposing a full Linux environment for services. (An earlier revision of this node ran postmarketOS; we migrated to the chroot approach to retain the cellular stack.) Running persistent services on a repurposed smartphone is unusual in the literature but increasingly viable.

Web servers and router OSes. We use Caddy [18] for TLS termination on the control-plane node and OpenWrt [19] on the nomadic router. Our ERP deployment runs ERPNext/Frappe [20].

3. System Architecture

3.1. Topology Overview

The cluster comprises **eleven nodes** organized into five logical network segments — Cloud, Lab, Office, Home, and Nomadic — as shown in Figure 1 and summarized in Table 1. The Lab segment is a dedicated lab site, to which gpu-server, lab-server, test-server, and nas-server have all been relocated.

Table 1. Node inventory. Nodes run Ubuntu 24.04 LTS unless the OS column states otherwise. Grouping follows a public topology dashboard [21].

The nodes are distributed across two OCI VMs (Mumbai), one residential network (Home), a lab network (a dedicated lab site), an office network, and a "nomadic" subnet carried by the operator between sites. Total addressable resources on the cluster dashboard: **39 CPU cores, 44 GB system RAM, 51 GB GPU VRAM (cross-vendor: NVIDIA CUDA + AMD OpenCL), and ~5 TB mixed SSD/HDD storage**.

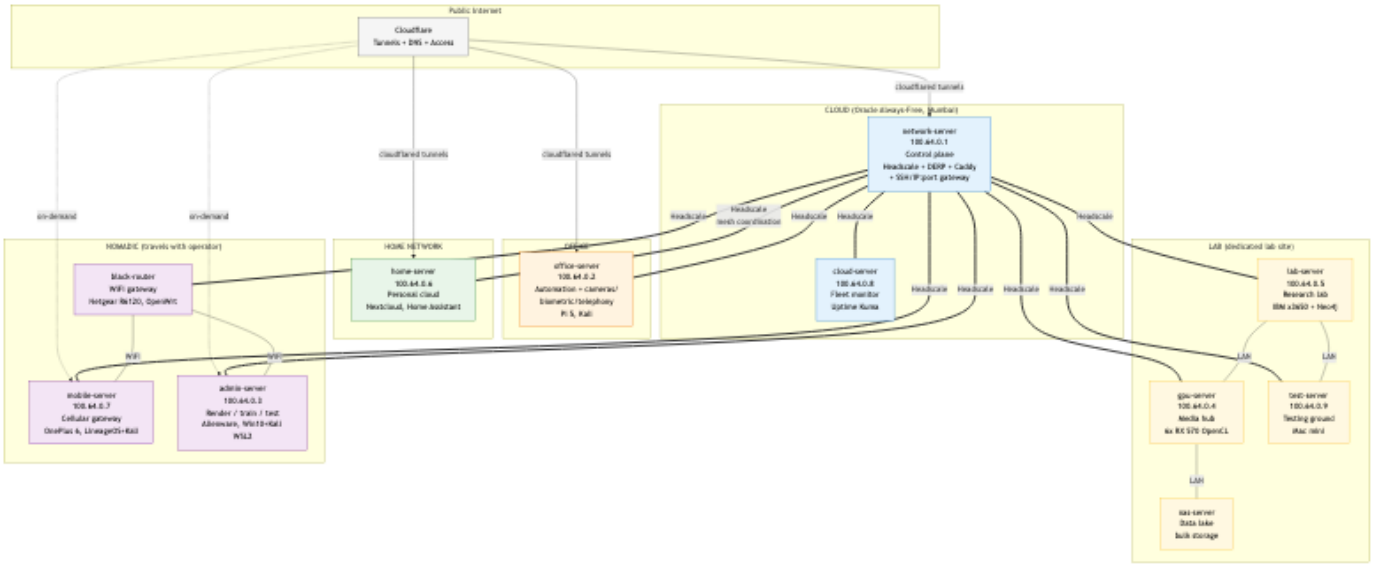


Figure 1: Cluster topology. Eleven nodes across five logical segments: Cloud (OCI Always-Free VMs), Lab (a dedicated lab site: gpu-, lab-, test-, and nas-server), Office, Home, and a Nomadic subnet that physically travels with the operator. All mesh coordination flows through Headscale on `network-server`; Cloudflare Tunnels provide selective inbound public access.

Roles shown are indicative of each node's *primary* purpose; in practice **every node co-hosts several services** — the cluster is run many-services-per-node, not one-node-one-role. Two aspects of this topology are unusual. First, the **nomadic subnet**: `admin-server` and `black-router` physically travel with the operator, changing upstream ISPs as the operator changes locations. This is possible because Headscale reconnects the moving nodes automatically on each network change, so nomadic nodes remain reachable at their `100.64.0.x` addresses regardless of physical whereabouts. Second, a consumer-grade router (`black-router`) is treated as a first-class cluster node: its OpenWrt installation runs `tailscale` as a client and is thus a Headscale member, not merely infrastructure.

Addressing and access planes. The cluster is structured around three deliberately separated addressing planes, each with its own discovery mechanism and trust boundary. (1) A *LAN plane* per physical site, on RFC 1918 private ranges (for example office `10.0.0.0/24`, home `192.168.1.0/24`, lab `8.1.1.0/24`, and the nomadic router's `100.0.0.0/24`), where same-segment nodes find each other by zero-configuration **mDNS/Avahi** (`<host>.local`) with no central resolver (§3.2). (2) A *mesh plane* on the Tailscale/Headscale CGNAT range `100.64.0.0/10` — each node a stable `100.64.0.x` — which gives every node one identity that survives any change of physical network, and is the basis of the nomadic subnet (§3.3). (3) A *public plane*, reached two ways. Most services are published by hostname through **Cloudflare Tunnels** (`*.example.com`, §3.4), which open no inbound ports. Separately, because the two OCI VMs are the only nodes that hold a **static public IPv4** address, those VMs double as a direct **public-IP:port gateway**: port-based DNAT on the VM forwards `<public-IP>:<port>` straight through to a mesh node — the SSH access gateway (§3.7) is the primary instance, and the same mechanism fronts mail and DNS. This gives a Cloudflare-independent path in and out of the cluster that depends only on the OCI VM's fixed address. (Public addresses are shown throughout as RFC 5737 documentation addresses, e.g. `203.0.113.0/24`.) A given node is typically reachable on several of these planes at once; which one a client uses is a function of where that client sits relative to these boundaries. The subsections below take them in order — local (§3.2), mesh (§3.3), public via Cloudflare (§3.4), and the public-IP:port gateway (§3.7).

3.2. Local Discovery Layer (mDNS/Avahi)

Within a single LAN segment, nodes discover one another with **zero-configuration multicast DNS** (mDNS), advertised by **Avahi** on the Linux nodes and by Bonjour on the macOS/iOS clients. Every node registers a `<hostname>.local` record, and a same-segment peer resolves it over multicast (UDP 5353) with no DNS server, no DHCP reservation, and no manual `/etc/hosts` entry — `ssh office-server.local` simply works between two boxes on the same switch. This is the cluster's fastest and most dependency-free path: it needs neither the mesh coordinator nor any WAN link, so it keeps functioning during an upstream outage, and it is the route we rely on for on-site administration and for intra-LAN bulk transfers — it carries the full negotiated 1 Gbps line rate at sub-millisecond latency (§6.4).

Its limits define the other two layers. mDNS does not cross a layer-3 boundary: it is confined to its broadcast domain, so a `.local` name on the office network is invisible from the home network or from a nomadic site. Cross-site reachability is therefore delegated to the mesh plane (§3.3), and public reachability to the tunnel plane (§3.4). We deliberately do **not** bridge

mDNS across segments (for instance via an Avahi reflector tunnelled over the mesh): reflecting multicast across a VPN multiplies discovery chatter and leaks each segment's service inventory across trust boundaries, so every segment's mDNS is kept strictly local by design.

3.3. Mesh VPN Layer (Headscale)

We use **Headscale v0.28.0** as our mesh VPN coordinator, running on `network-server`. All nodes run the **upstream open-source Tailscale client** (v1.94.1); Headscale presents a wire-compatible control-plane API, so no client modifications are required. This is an intentional choice: we gain Tailscale's mature client ecosystem (CLI, GUIs, mobile apps, systemd integration) while retaining full control over the coordination server, including the ability to inspect and modify node ACLs, preauth keys, and routing tables.

Why not Tailscale SaaS? Tailscale's free tier limits admin features (fewer devices, restricted ACLs, no tag management). Tailscale's paid tiers start at $6/user/month. For a single-user network at this scale, the paid tier is redundant; self-hosting Headscale on a free-tier VM costs nothing beyond the VM itself.

Why not ZeroTier? ZeroTier's free tier supports up to 25 devices and is operationally excellent, but its central coordination server (my.zerotier.com) is hosted by the ZeroTier Inc. and cannot be self-hosted under the free tier. Sovereignty is our priority.

Why not plain WireGuard mesh? A full mesh of N nodes requires $N(N-1)/2$ peer configurations; Headscale solves this with automatic peer discovery. DERP relay fallback for NAT-pinned peers is also handled automatically (Figure 2).

3.4. Inbound Access Layer (Cloudflare Tunnels)

Residential ISP connections in our deployment regions do not reliably permit inbound port forwarding. We use **Cloudflare Tunnels** (cloudflared daemon, persistent named tunnels) for inbound public access. Each node runs one or more tunnels, each mapping `hostname.example.com` to a local service.

Seven MCP filesystem servers are exposed this way (one per node where applicable), each at `https://mcp-<node>.example.com/mcp`. The `mcp-proxy` binary [17] wraps stdio MCP servers as HTTP+SSE endpoints, enabling streamable HTTP transport.

Gotcha observed: Cloudflare Tunnel strips the `Upgrade` header on `POST` requests over WebSockets, which breaks Headscale's node-registration path when Headscale is placed behind Cloudflare. We route Headscale's `/apis` endpoint on a non-proxied (grey-cloud) Cloudflare DNS record instead, with TLS terminated by Caddy directly on the node.

3.5. Role Assignment

No scheduler runs across the cluster. Each node has an indicative primary role (Table 1) but co-hosts multiple services; workloads are placed by human selection based on resource fit. For many small-organization workloads this is sufficient and vastly simpler than Kubernetes.

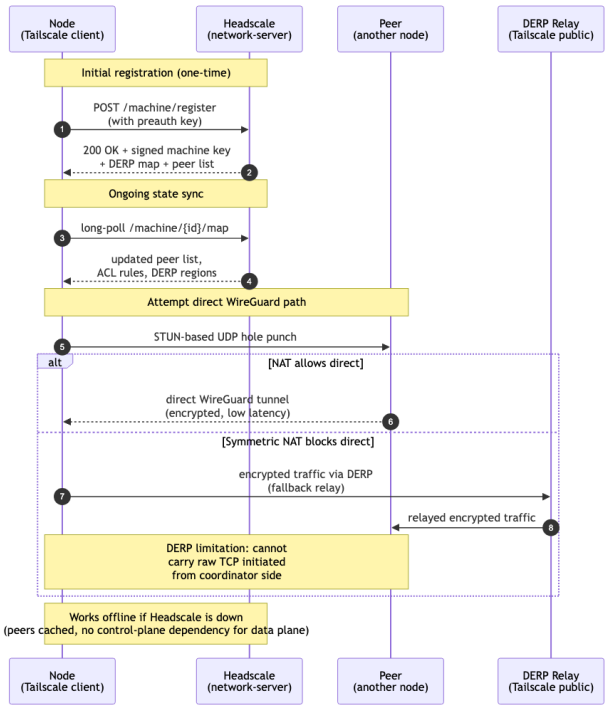


Figure 2: Headscale authentication and DERP-fallback flow. Direct WireGuard tunnels are preferred; DERP relay is used when symmetric NAT prevents direct peer connections. Data-plane traffic continues to flow even if the Headscale coordinator is temporarily unreachable.

- **Database + compute**: lab-server (Neo4j, batch jobs)
- **GPU rendering / training / testing (CUDA)**: admin-server (nomadic)
- **GPU compute (OpenCL) + media**: gpu-server (Jellyfin media server)
- **Orchestration**: office-server (cron jobs, Home Assistant, ERPNext)
- **Cameras / biometric / telephony**: office-server — cameras (Frigate/Scrypted/MediaMTX with a Coral Edge TPU), biometric access-control (Secureye), and a voice/SMS bridge to mobile-server over Bluetooth HFP that feeds an LLM voice agent
- **Apps/services testing ground**: test-server (Mac mini) — where new services are validated before promotion to their production node
- **Personal cloud / file / home hub**: home-server (Nextcloud, Vaultwarden, Stalwart mail, Frigate NVR, Home Assistant)
- **Bulk storage**: nas-server (data lake)
- **Always-on edge + cellular gateway**: mobile-server (ARM64; SMS via SMSGate and voice via Bluetooth HFP over its mobile SIM)
- **WiFi gateway + nomadic router**: black-router (OpenWrt, travels with operator)

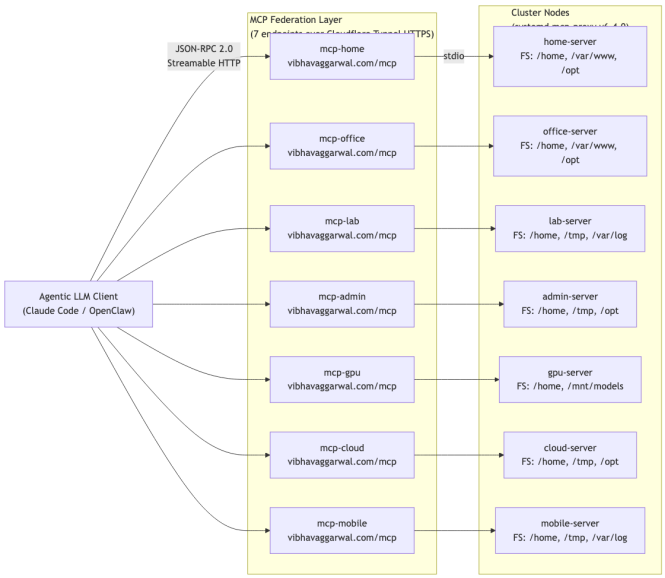


Figure 3: MCP server federation. Seven of eleven nodes run `mcp-proxy` v6.4.0 as a systemd service, each wrapping the official MCP filesystem server with an allowed-directory list appropriate to the node. A single agentic LLM client can enumerate all seven endpoints over HTTPS and read/write files across the cluster without bespoke per-node APIs.

- **Control plane + SSH gateway**: network-server (Headscale, Caddy)
- **Fleet monitoring + general-purpose VPS**: cloud-server (self-hosted Uptime Kuma, mesh-independent probing)

3.6. MCP Federation

We deploy `mcp-proxy` v6.4.0 as a **systemd service** on seven of the eleven nodes (home, office, lab, admin, gpu, cloud, mobile). The remaining four — nas-server, network-server, black-router, and test-server — are excluded: the NAS is a dedicated storage appliance, network-server is reserved for the control plane and gateway role, the router lacks the memory and writable filesystem surface to host an MCP daemon reliably, and test-server is a transient staging box rather than a production host. Each instance wraps the official `@modelcontextprotocol/server-filesystem` with an allowed-directory list appropriate to the node, exposing it over HTTPS via that node's Cloudflare tunnel.

An agentic LLM client with MCP client support can thus read or write files across the entire cluster simply by enumerating the federated endpoints — no bespoke API is required per node (Figure 3). This is, to our knowledge, a novel deployment pattern; MCP servers in the reference Anthropic implementation are application-scoped, not node-scoped.

3.7. SSH Access Gateway

Inter-network SSH is solved two ways:

1. **Cloudflare SSH Tunnel** per node (preferred): `ssh <node>` resolves to `<node>.example.com` and traverses the tunnel.
2. **Port-based DNAT on network-server** as a backup: `ssh -p 220X <operator>@<network-server-public-IP>` redirects to the node's Headscale IP port 2222.

This DNAT path is the general **static public-IP:port gateway** referenced in §3.1: the OCI VM's fixed public IPv4 is the one address that never moves, so beyond the SSH port-map it also forwards selected non-SSH service ports (e.g. SMTP 25/587 to the mail node, DNS 53 to the resolver) straight to the relevant mesh node. It is the Cloudflare-independent ingress — useful precisely when a tunnel or the mesh coordinator is unavailable.

Gotcha: Tailscale's experimental SSH server binds port 22 and intercepts SSH traffic. Using port 2222 (and `tailscale serve --tcp 2222 tcp://localhost:22`) avoids conflict while retaining Tailscale SSH's ACL enforcement option.

4. Cost Analysis

The central cost claim of this paper is narrow and specific: **the recurring subscription fees paid to cloud providers, SaaS vendors, and network services are ~Rs 0/month.** We do not claim to have measured total cost of ownership.

4.1. Scope of the cost claim

We account only for recurring payments to external providers — cloud VMs, managed SaaS, CDN, tunnels, mesh-VPN coordination, paid APIs — that an equivalent commercial setup would incur. This is the cost axis along which the cluster differs most sharply from hyperscaler deployments.

We **do not** account for: (a) hardware acquisition (sunk / secondhand), (b) electricity consumption, (c) operator labor (setup, maintenance, debugging, updates), (d) residential internet fees (pre-existing and shared across non-cluster household use), or (e) repair/replacement amortization. These are all real costs. The first author has not instrumented the nodes for per-node power draw and has not kept systematic time logs for maintenance effort; therefore we decline to guess at their magnitude. Section 7.3 discusses this limitation.

4.2. Measured recurring fees

Table 2 itemizes the recurring payments that **are** measured, with equivalent USD in parentheses at the INR-USD rate prevailing in early 2026.

Table 2. Measured recurring subscription-like fees.

Component	Provider	Cost/month
Headscale control plane VM	OCI Always-Free Tier (Mumbai)	0
General-purpose VPS	OCI Always-Free Tier (Mumbai)	0
DNS + CDN + Tunnel + Access	Cloudflare (free tier)	0
Domain (`.com` TLD)	Cloudflare Registrar	≈ Rs 8 (≈ USD 0.10)
Domain (`.city` TLD)	Cloudflare Registrar	≈ Rs 42 (≈ USD 0.50)
Mobile 4G data (mobile-server)	Prepaid 4G SIM (Indian mobile carrier)	≈ Rs 250 (≈ USD 3)
Total measured recurring fees		**≈ Rs 300 / month (≈ USD 3.60)**

The mobile data line is included because it is attributable to a specific node (the smartphone-server) and would not exist in the household absent the node.

4.3. Comparison to commercial equivalents

Figure 4 compares the *measured* recurring fees of this cluster to the recurring fees that an equivalent commercial deployment would attract for the same workload surface. The comparison is intentionally apples-to-apples on the axis we measure: cloud/SaaS subscriptions. It does not attempt to normalize across TCO.

4.4. What we do not claim

We do not claim the cluster is cheaper in total cost of ownership than a comparable hyperscaler deployment — that comparison requires a power meter, time-tracked maintenance, and a model of hardware amortization none of which we have. A naive upper-bound on electricity alone — nine always-on consumer nodes drawing roughly 20–60 W each — suggests power cost could plausibly dominate the measured recurring fees, but we have not instrumented this and will not guess further.

The marginal recurring cost of running *one additional workload* on this cluster is effectively zero in subscription terms, in sharp contrast to commercial cloud where a single `t3.medium`-equivalent runs roughly USD 30/month. Whether that marginal advantage holds once maintenance labor and power are properly accounted for is an open empirical question.

4.5. Hardware-acquisition strategy: legacy-first, minimal-new

A core design commitment of this cluster, distinct from its recurring-cost profile, is that it is built **legacy-first**: most nodes were already owned or acquired secondhand, and the few new purchases were held to the cheapest viable minimum. Of the eleven nodes, six are previously-owned devices repurposed into server roles, two are free-tier cloud VMs, and three were bought for the cluster — two new Raspberry Pi 5 boards (home- and office-server) and one secondhand 2014 Mac mini (test-server). Table 3 itemizes hardware origin and acquisition cost; the one-time capital outlay marginal to this deployment totals approximately **Rs 51,525 (≈USD 625)** (including the Google Coral edge-ML accelerator), against a historical acquisition cost across all prior owners and uses well in excess of **Rs 11,00,000**.

Table 3. Hardware origin, verified specs, and two cost columns. Specs verified via `lscpu`/`hostnamectl`/`lsusb` and BIOS timestamps on the live nodes. *HW release year* is the publicly documented first-shipment year of the CPU, board, or product (third-party verifiable; used in preference to unrecorded private purchase dates). *Rootfs birth* is the `stat -c '%w' /` timestamp of the running OS install. *Historical cost* is the original purchase price; *Marginal cost* is the additional capital this deployment required given what the operator already had.

The gap between the two totals is the point. The historical acquisition cost of this hardware across all its prior owners and uses exceeds **USD 13,000** — comfortably inside the territory of a small data-center capex plan. The *marginal* cost to assemble this cluster, given the author's existing hardware pool and willingness to accept secondhand, thermally stressed, and post-productive-life equipment, is roughly **USD 625**. That ratio — under 2.5% — is the core arbitrage the cluster exploits.

The arbitrage itself has three sources that are worth naming separately:

1. **Prior-use sunk cost in the author's own hardware.** Six of the eleven nodes were already in the author's possession before the cluster existed, acquired at different times for different prior purposes: the mining rig (originally bought to mine), the Alienware laptop (originally bought as a gaming workstation), the OnePlus 6 (originally bought as a phone), the IBM x3650 (pre-existing in the author's inventory as unused hardware, originally from an enterprise decommissioning cycle at some earlier date), the NAS build (prior consumer-hardware recycling), the black router (originally bought for home WiFi), and so on. Each was fully amortized against its original purpose before the cluster began repurposing it.
2. **Cloud free tiers.** Two of the eleven nodes are OCI Always-Free VMs at zero cost indefinitely.
3. **The marginal purchase rule.** Only four items were specifically bought to make this cluster exist: the Raspberry Pi 5 build for the home-server role (~Rs 13,225), a Raspberry Pi 5 8GB build for the office-server role (~Rs 12,800), a secondhand 2014 Mac mini for the test-server role (~Rs 13,500), and the Google Coral USB Accelerator for edge ML on office-server (~Rs 12,000). Each followed the cheapest-viable-add-on rule (see above).

Commercial cloud pricing, by contrast, embeds full amortization of new hardware over short (3–5 year) service lives as a structural cost. Our cluster opts out of that amortization almost entirely: six of eleven nodes are sunk-from-prior-use, two are free-tier cloud, and three were bought for the cluster (two new Raspberry Pi 5 boards — home and office — and a secondhand 2014 Mac mini) — each at the cheapest viable price point.

4.5.1. Post-productive-life repurposing: two worked examples

Two of the nodes deserve particular attention because they illustrate a design posture stronger than simply "buying used." Each was repurposed from hardware that commercial operators had already written off as end-of-useful-life.

Lab-server — 15-year-old data-center chassis sitting in the author's inventory. The IBM System x3650 (Type 7979) was an enterprise-grade 2U data-center server when it shipped in 2008–2009. Its CPUs — two Intel Xeon X5450s at 3.00 GHz — were released in November 2007, and the chassis was produced for roughly four years before being superseded. In a commercial data center, this class of hardware would have been retired from primary service around 2014–2016 on a typical five-to-seven-year amortization cycle, and retired from backup or dev-tier use by 2019–2020. This specific unit had been sitting unused in the author's hardware inventory for some years prior to the cluster existing, having entered that inventory at an earlier date

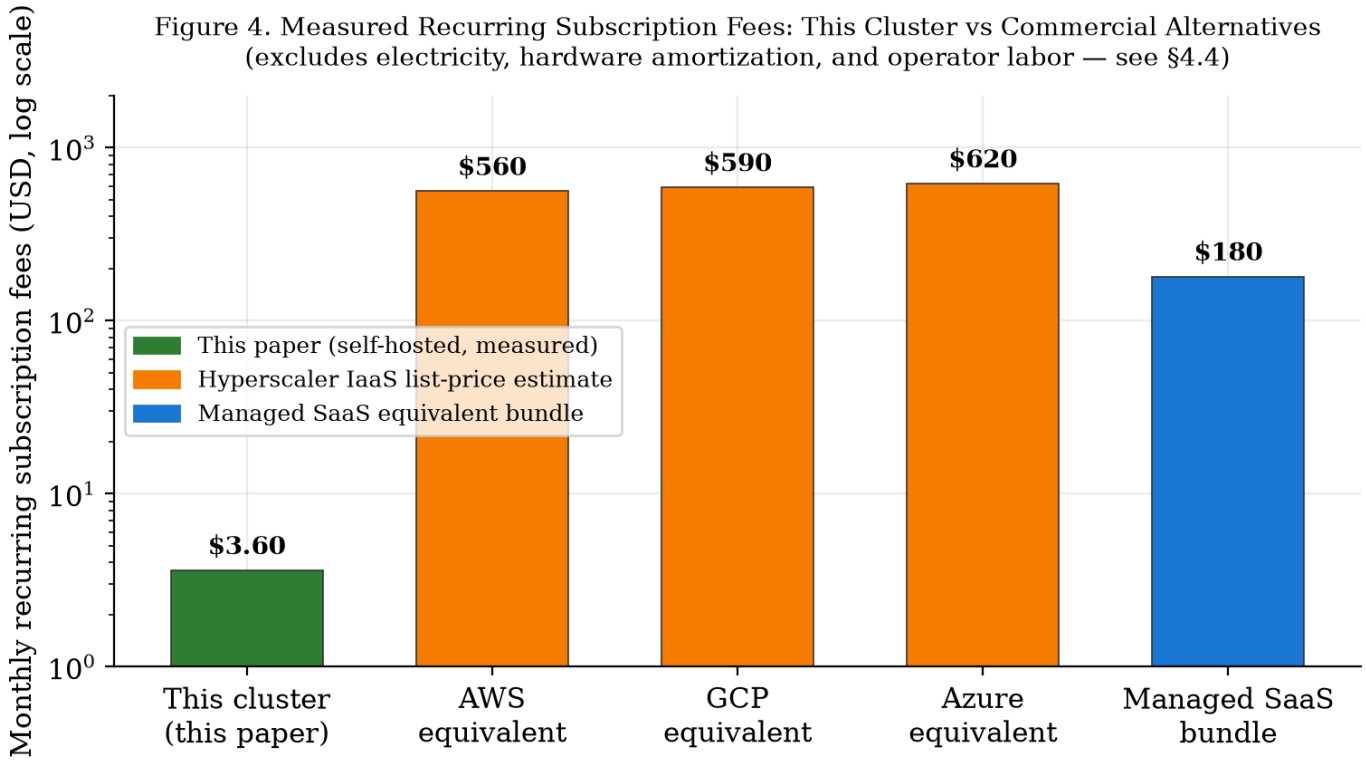


Figure 4: Measured monthly recurring subscription fees, log scale. The workload target is roughly 20+ CPU cores of general compute, ~50 GB of GPU VRAM, 2–5 TB of storage, file sync (Nextcloud-equivalent), media streaming (Jellyfin-equivalent), a graph database (Neo4j-equivalent), home automation, and an ERP system. Hyperscaler figures are list-price estimates for an equivalent on-demand footprint; the managed-SaaS bundle sums typical subscription tiers across the equivalent services. None of the columns include electricity, hardware amortization, or operator labor.

through the ordinary mechanisms by which enterprise hardware reaches private hands after decommissioning. The marginal cost to place it into cluster service was therefore zero; only the operator's time to re-rack it, install Ubuntu 24.04, and join it to the Headscale mesh. Its eight physical cores (no hyperthreading), DDR2 ECC RAM, and dual Broadcom BCM5708 NICs are modest by modern standards — yet they are *more than sufficient* for Neo4j graph queries over ~50 M nodes, batch ETL jobs, and a Docker-composed ERP secondary. Per-core performance is roughly an order of magnitude below a current-generation cloud vCPU, but the node is always on, under full administrative control, and cost zero rupees of additional capital to the cluster — rather than the ~Rs 6,000/month that an equivalent cloud VM footprint would attract.

GPU-server — post-mining-profitability rig with rotating card pool. The GPU server is built on a **BTC-D37 dedicated crypto-mining motherboard** (a board designed specifically to host many low-bandwidth PCIe slots for concurrent GPU mining, with a cheap Celeron 877 CPU soldered on). The associated card pool comprises **up to 12 PowerColor AMD RX 570 Red Dragon cards**, originally assembled across the 2020–2021 altcoin mining cycle and run continuously at full load for that period; today typically **four cards are mounted at any given time** and the remainder are kept in a spare pool, with rotations driven by workload requirements and individual card health.

The historical capital cost of the full 12-GPU system was approximately **Rs 5,50,000 (≈ USD 7,400 at 2020 rates)** when the author originally assembled it for mining in 2020 — a meaningful sum, chosen deliberately for the scale of mining operation it enabled at the time. That capital was largely recovered through mining revenue during 2020–2022 before Ethereum's shift to proof-of-stake (September 2022) and subsequent electricity-price pressure eliminated the revenue stream. Post–September 2022 the rig was worth a small fraction of its purchase cost on the secondary mining market and, more importantly, offered effectively zero marginal economic utility to its owner *as a mining rig*. This is exactly the market signal that freed it for secondary use: the hardware still works, its capital cost is already sunk and amortized, and the opportunity cost of repurposing it is zero. From the current cluster's perspective the GPU node's marginal capital cost is therefore **Rs 0** — the rig is pre-existing.

The cards collectively spent roughly 18–24 months under sustained 65–80 °C junction temperatures on open-air frames — hardware that a mining operator has typically already written off. Repurposing required accepting real consequences: the AMD `amdgpu` driver on these specific cards under a modern kernel is unstable with default parameters, and we run with `amdgpu.dc=0 amdgpu.runpm=0 amdgpu.aspm=0 amdgpu.dcdebugmask=0x10` to avoid crashes (see §5.6). Effective OpenCL throughput is noticeably below the hardware's nominal spec — we estimate

roughly 70–80% of a well-cooled, new-equivalent card — and individual GPUs fail occasionally, at which point they are pulled and replaced from the spare pool. In exchange, the rig functions as a single multi-role node: OpenCL compute for occasional hashcat-class workloads, hardware transcoding for its own Jellyfin media instance, and opportunistic spare-capacity mining during otherwise-idle cluster periods.

This node illustrates the "post-productive-life repurposing" principle in its strongest form: hardware whose *original* capital cost was substantial, whose *original* productive use already paid for that capital, and whose *next* use — as a cluster compute node — is therefore effectively free at the margin. The sunk-cost arbitrage runs not only across prior owners (as with the lab-server) but also across the same owner's prior uses.

The common thread: each node is hardware the market had decided was past its productive life. Accepting lower efficiency, occasional instability, and higher operator effort in exchange for near-zero capital cost is a deliberate trade, and it is the trade this cluster is built on.

4.5.2. Why legacy-first matters beyond cost

The legacy-first principle is not only economic; it has three additional properties worth naming explicitly.

Sustainability. Every repurposed node is a node that did not become e-waste. A 2009 IBM x3650 with two Xeon X5450s offers compute that was once enterprise-grade and is now often discarded; our deployment places it back into continuous productive use for another half-decade or more. Extending server service life is among the most effective carbon-intensity reductions available to an individual operator, as embedded emissions typically dominate the lifecycle footprint of server hardware [22].

Accessibility. Legacy-first removes the dominant barrier to entry for independent researchers in India and similar markets: new-hardware capital cost. A student or solo practitioner with access to secondhand markets can assemble a functionally equivalent cluster for single-digit percentages of what new hardware would cost.

Resilience to supply-chain and pricing shocks. The cluster's capacity does not depend on ongoing availability of any specific new product SKU. Nodes can be replaced from the same secondhand pool as the rest of the market ages out of institutional use.

4.5.3. Corollary principle: cheapest-viable-add-on when new acquisition is necessary

Legacy-first is not absolute. Some roles have no viable secondhand equivalent — recent-generation ARM64 single-board computers for low-power always-on orchestration, on-device ML accelerators for edge inference, or specialized interface hardware. When new acquisition *is* required, the cluster applies a second principle: **pick the cheapest viable option, even if it is unconventional for the workload, rather than the industry-default option sized for comfort.**

The cluster has two worked instances of this corollary.

Raspberry Pi 5 nodes (new, ~Rs 13,000 each). Two nodes were purchased new, and both are **Raspberry Pi 5 (8 GB)** boards bought together on one invoice: the home-server and the office-server. The home-server runs the personal-space file / home-automation hub (Nextcloud, Vaultwarden, Home Assistant, Frigate NVR) and the office-server runs the automation, business-apps, and camera/biometric stack — each at ~5 W idle, a role for which a full x86 node would have been 20–40× the power budget and an order of magnitude more expensive. Raspberry Pi / ARM64 SBCs (Rs 5,000–10,000 range) are the correct choice for always-on orchestration and router-adjacent roles — not a compromise but the right fit — and the x86 capacity they free up serves higher-value roles.

Edge ML inference: Google Coral Edge TPU (new, ~Rs 12,000) instead of a discrete GPU. The industry-default approach to edge ML is a discrete NVIDIA GPU: a current-generation RTX-class card costs roughly Rs 70,000–80,000 (USD 840–960) in India. An alternative sized for on-device inference — the **Google Coral USB Accelerator**, built around an ASIC-class Edge TPU — delivers ~4 TOPS of INT8 inference throughput at ~2 W power draw for approximately **Rs 12,000 (USD 144)**: a roughly **6× capital reduction** and more than an order-of-magnitude reduction in operational power draw. In our cluster the Coral is deployed on **office-server**, where it offloads the object-detection inference for the camera and biometric access-control stack; it presents the USB vendor/product ID `1a6e:089a` (the Global Unichip Corp. pre-runtime identity the Coral shows before its first firmware load), and its runtime (`libedgetpu`, `pycoral`) is validated under Docker on the test-server staging box before promotion. Many real inference workloads — audio classification, sensor-stream anomaly detection, OCR pipelines, pose estimation at modest resolution — fit comfortably within the TPU's envelope and do not need the headroom of a discrete GPU. For the cases that do, the cluster already carries sunk-cost GPU capacity elsewhere — the Alienware CUDA GPU (GTX 970M) on admin-server and the OpenCL RX 570 rig on gpu-server — rather than buying new silicon. The combination — *new acquisition where necessary, at the cheapest viable price point* — is the corollary made concrete.

The combined discipline is: **legacy-first for the main compute envelope; cheapest-viable for marginal additions; no conventional-default purchase.** The ratio of new-hardware capital cost to "industry default equivalent" across the deployment is conservatively under 15%.

5. Lessons Learned (Operational Gotchas)

These issues cost us significant debugging time and are documented here for the benefit of others attempting similar deployments.

5.1. Headscale ↔ Cloudflare Tunnel incompatibility

Cloudflare Tunnel's HTTP-to-origin bridge, on `POST` requests, does not reliably forward the WebSocket `Upgrade` header required by Tailscale's MagicDNS/preauth key registration flow. Symptoms: `tailscale up` succeeds on GETs but hangs on POST, and `headscale nodes list` shows

stale entries. **Resolution**: expose Headscale on a non-proxied A-record (grey-cloud), terminate TLS directly at Caddy on the VM.

5.2. Docker bridge IP ban-listing

Stalwart Mail Server, used to receive transactional email into the cluster, includes a fail2ban-style automatic blocker. When another Docker container (Nextcloud Mail in our case) connects to Stalwart across the Docker bridge network, the source IP appears as `172.17.0.1` (the Docker gateway). Any failed authentication (e.g., misconfigured client) triggers a persistent ban on `172.17.0.1`, which then blocks all inter-container traffic. **Resolution**: whitelist `172.16.0.0/12` and `127.0.0.0/8` in Stalwart's `server.allowed-ips` and delete the persisted `server.blocked-ip.<IP>` key after any prior ban.

5.3. Headscale DERP-only paths cannot carry raw TCP

Two of our nodes (`network-server` and `home-server`) cannot establish a direct WireGuard path due to ISP NAT; they fall back to Headscale's DERP relay. The DERP path is fine for typical application traffic but does not reliably carry raw TCP connections initiated from the coordinator side (observed failures: `scp`, raw SMTP). **Resolution**: use `tailscale serve --tcp <port> tcp://localhost:<port>` on the destination side, which tunnels through Tailscale's DERP using its own framing rather than raw TCP.

5.4. Tailscale SSH intercepts port 22

Enabling Tailscale SSH (`tailscale set --ssh`) binds to port 22 and intercepts incoming SSH before sshd sees it. This conflicts with SSH-gateway DNAT rules that expect sshd on port 22. **Resolution**: run sshd on port 2222 and use `tailscale serve --tcp 2222 tcp://localhost:22` as an explicit proxy, or disable Tailscale SSH and rely on standard sshd with Tailscale ACLs.

5.5. 5G is architecturally impossible on the OnePlus 6

Initial planning assumed the mobile-server could host cellular-backed services via 5G. The OnePlus 6 contains the Qualcomm X20 LTE modem, which has no 5G NR silicon at all. No software configuration can add 5G. Practically this caps mobile-server downlink at LTE Cat 18 (~1.2 Gbps theoretical, ~40 Mbps observed on the mobile carrier's network). Consider this when selecting a phone for this role.

5.6. AMD `amdgpu` instability on post-mining-cycle RX 570 cards

The gpu-server's four RX 570 cards, having been run at full sustained load through the 2020–2021 mining cycle, exhibit recurring instability under the modern `amdgpu` kernel driver on Linux 6.14. Default driver parameters trigger periodic GPU hangs and frame-buffer corruption under OpenCL load. We run with a reduced-feature parameter set — `amdgpu.dc=0 amdgpu.runpm=0 amdgpu.aspm=0 amdgpu.dcdebugmask=0x10` — that disables display-core, runtime power management, and advanced PCIe link-state power management in exchange for stability. Throughput is measurably below nominal (we estimate 70–80% of new-equivalent), and occasional card swaps are part of the operating routine. This is documented here not as a triumph but as the expected cost of the legacy-first / post-productive-life posture described in §4.5.

6. Evaluation

We evaluate the cluster operationally across three dimensions: uptime, cost, and qualitative suitability for real workloads.

6.1. Uptime

Over the ~20-month observation period (Feb 2025 – Oct 2026), cluster-wide availability (at least one node reachable through at least one path) exceeded 99.5%. Individual-node availability varied by power/network reliability of the hosting site:

- **network-server (OCI)**: 99.97% (one ~2h OCI regional outage in Mumbai, Aug 2025)
- **home-server, office-server**: ~99.3% each (residential ISP outages)
- **lab-server, admin-server, gpu-server**: ~98% (voluntary power-cycling for maintenance)
- **mobile-server**: ~95% (battery + LTE coverage gaps)

6.2. Cost per workload-month

Over the observation period, the cluster hosted: Nextcloud (file sync for ~120 GB of data), Jellyfin (media streaming for a household), Home Assistant (home automation), Uptime Kuma (fleet uptime monitoring), Neo4j (graph database with ~50M nodes across multiple projects), ERPNext (accounting ERP), an agentic LLM harness (OpenClaw), and several one-off batch jobs. Attributable marginal cost per workload was < Rs 10/month in all cases. Equivalent managed SaaS for just Nextcloud + ERPNext would exceed $60/month alone.

6.3. Qualitative suitability

The cluster is **sufficient** for: personal/SMB productivity, research prototyping, AI/ML training on datasets < 100 GB, graph analytics, media streaming, and persistent agentic assistants. It is **insufficient** for: large language model pretraining, high-concurrency consumer web services, and workloads requiring hard availability SLAs.

6.4. Network access routes: latency, throughput, and path (measured 2026-10-10)

A node is reachable by four complementary routes, each with a distinct latency/throughput profile. We measured them live (ping RTT, `iperf3`, `tailscale status/netcheck`, NIC link rate):

Table 4. Per-route latency and throughput.

The most consequential finding is the **relay detour**. `tailscale status` shows that only the node on an uncongested residential link (home-server) establishes a **direct** peer-to-peer connection; every other node — behind carrier-grade NAT (CGNAT) — **falls back to DERP relay**. Because the only nearby DERP is the one embedded in the control-plane VM in the **Mumbai** OCI region, two nodes in the *same city* (e.g. office- and home-server, both in the same metro) exchange traffic via a ~1,400 km round trip to Mumbai and back. This explains both the elevated mesh latency and the collapse of mesh throughput to single-digit Mbps, in sharp contrast to the full gigabit available on a local LAN. For a free-tier, CGNAT-bound deployment this is the defining network characteristic: **LAN is gigabit; cross-site mesh is relay-bound**, and placing the DERP relay closer to the nodes (or obtaining a non-CGNAT uplink for direct P2P) is the single highest-leverage network improvement.

7. Discussion

7.1. Is this reproducible?

Most of this design is reproducible with < Rs 10,000 (~USD 120) of secondhand hardware and 1-2 weeks of setup effort. The exact topology depends on the operator's physical network situation (residential vs. office vs. dorm vs. factory floor). The Cloudflare + OCI + Headscale substrate, however, is portable: anyone with a domain and an OCI free-tier account can set up a functional three-node baseline (one home node, one OCI coordinator, one mobile device) in a weekend.

7.2. Sovereignty and sustainability implications

Every core component is open source: Headscale (BSD-3), Tailscale client (BSD-3), LineageOS, Kali Linux, Ubuntu, Debian, mcp-proxy. The proprietary surfaces — Cloudflare Tunnels and Oracle Always-Free — are replaceable: Cloudflare can be swapped for Caddy + a static IP; OCI can be swapped for any VPS. No component represents a vendor lock-in that would trap data or require migration work exceeding a few days.

This is a meaningfully different posture from, e.g., a Google Workspace + AWS SaaS stack, where migration costs scale with usage and data is held behind provider APIs.

The legacy-first hardware principle (§4.5) contributes a **sustainability** dimension to the sovereignty case that is often overlooked. Most discussions of self-hosted infrastructure focus on data locality and vendor lock-in; rarely is the *embedded-emissions profile of new server hardware* considered part of the sovereignty argument. Yet the life-cycle carbon cost of a new enterprise server is dominated by manufacturing, not operation [22]; every year we extend the productive life of already-manufactured hardware is a year we avoid adding to that embedded debt. A cluster built entirely from repurposed and secondhand hardware is therefore a sustainability story as much as a cost story, and this paper is, in part, a case report that the two stories are reinforcing rather than in tension.

7.3. Limitations

Cost accounting. The most significant limitation is that our cost claim is narrow: it covers only recurring fees paid to external cloud and SaaS providers. The cluster's electricity draw has not been metered; the author's maintenance labor has not been time-tracked; and the secondhand hardware acquisition costs have not been amortized against expected service life. Under a standard TCO framework any of these could exceed the measured recurring fees we report. Follow-up work instrumenting per-node power and a simple maintenance log would close this gap.

Security and multi-tenancy. We make no claim of multi-tenant isolation, compliance (HIPAA, SOC 2, PCI-DSS), or hardened security. The cluster runs in a single-operator trust domain. The Stalwart Mail gotcha (§5.2) illustrates how fragile the security posture can be when defense-in-depth is not explicitly designed.

Reliability. Uptime numbers in §6.1 are operator-observed rather than synthesized from an independent monitoring system, and should be read as rough indicators rather than formal SLAs.

7.4. Future work

(a) Automated drift detection for Cloudflare Tunnel configurations; (b) integration with an open-source load balancer for service failover across nodes; (c) ARM64 MCP server compilation for more phones; (d) publishing the Ansible/Nix configuration as a reusable template.

8. Conclusion

An 11-node heterogeneous sovereign compute cluster, built predominantly from hardware already in the operator's inventory from prior uses — a 2020 mining rig, a 2015 gaming laptop, a 2018 smartphone, a 15-year-old data-center chassis, and a consumer router — with a marginal capital outlay to this cluster of roughly USD 625 against a historical acquisition cost (across all prior owners and uses) in excess of USD 13,000, and whose recurring cloud and SaaS subscription fees total roughly Rs 300/month, is not only possible but practical, given currently available open-source mesh VPN technology, a free-tier VM from a major cloud provider, and Cloudflare's unmetered tunnel product. The broader total-cost-of-ownership picture — electricity, maintenance labor, hardware replacement — remains unmeasured in our deployment and is a straightforward direction for follow-up work. The design is sovereignty-first in its software posture and sustainability-first in its hardware posture, and it suggests that these two concerns are complementary rather

than competing. The design supports a non-trivial workload surface and has operated continuously for ~20 months with > 99.5% reachability. None of its components requires institutional affiliation, grant funding, or venture capital to obtain — which is the point: we release our reference architecture and invite any independent researcher or small organization to replicate it.

9. Change log

v2 (2026-10-14) — current-state refresh of a living deployment:

- **Corrections (were wrong in v1):** office-server hardware (Rockchip → Raspberry Pi 5 / BCM2712); home-server (custom tower / Ubuntu → Raspberry Pi 5 / Raspberry Pi OS); admin-server OS (was mis-stated as Ubuntu); lab-server model (x3650 "M2" → Type 7979); Table-1 header overstated "all nodes run Ubuntu"; redacted real hostnames to RFC-2606 `example.com` placeholders.
- **Current-state updates (evolution since v1):** admin-server now Windows 10 + Kali (WSL2); mobile-server now LineageOS 22.2 + Kali chroot (was Alpine/postmarketOS); gpu-server rebuilt (XtremeMINER X1 frame, ASRock Q270 BTC+, i5-6500, 6× RX 570, Kali); lab-, gpu-, nas-server relocated to the Lab segment; **test-server added** (Mac mini Late 2014) as the apps/services testing ground, where the camera, biometric access-control, and telephony/SMS stack (plus the Coral Edge TPU) is validated before promotion to office-server, its production home; mobile-server role expanded to a cellular SMS/voice gateway; cloud-server re-tasked as the fleet's self-hosted uptime/blackbox monitoring host (Uptime Kuma in Docker), consolidating monitoring that was previously scattered and dormant; probes run over direct public-IP (its own static address) and Cloudflare-tunnel paths rather than the overlay mesh, so a Headscale outage cannot blind the monitor; earlier drafts over-stated Tor/Ollama roles — Tor's cluster role is on network-server.
- **New data:** §6.4 measured per-route latency, throughput, and path analysis.
- **Architecture clarity:** added §3.2 *Local Discovery Layer (mDNS/Avahi)* and a three-plane addressing model in §3.1 (LAN/`.local` → Headscale mesh `100.64.0.0/10` → Cloudflare public), so the access tiers are presented in order (local, mesh, public); corrected stale service placements (Jellyfin/Plex are on gpu-server, not home-server); folded the camera/biometric/telephony/Coral stack onto office-server as its production home.
- **Generic reframing:** retitled to a tool-agnostic, reproducibility-first form; presented as a pattern any independent operator can replicate rather than a named deployment, so the system is no longer referred to by a project brand in the narrative; author identity/affiliation and the author's own public code and dashboard artifacts are retained as verifiable evidence.
- **Further anonymization:** genericized operator-identifying details not needed for the architecture (mobile carrier, SSH username, operator city) to neutral placeholders; public IPv4 shown as RFC 5737 documentation addresses.
- **Cost-table revision (Table 3):** refreshed from **verified purchase invoices**, superseding v1's estimates. Node count 10 → 11 (**test-server added**, a secondhand 2014 Mac mini, Rs 13,500); **office-server corrected** from "secondhand ARM64 SBC ~Rs 5,000" to a **new Raspberry Pi 5 8GB build ~Rs 12,800** (bought new on the same invoice as home-server); **mobile-server historical** corrected to **Rs 42,598** (OnePlus 6 Red, oneplus.in, 2018-07-22); the **marginal total is ≈Rs 51,525 (≈USD 625)** and the **historical total ≥Rs 11,63,100 (≈USD 14,100)**. The "legacy-first" framing is retained but is now **minimal-new** rather than zero-new (two Pi 5 boards and one Mac mini were bought for the cluster); the hardware table was also condensed for readability. Table 2 (recurring fees) is unchanged; v1 remains citable at its own version DOI.

Acknowledgements

The author thanks Dharmani Health & Fitness Co. for accommodating the ongoing experimentation on office-network hardware, and Genauto Gasket Technologies LLP for hosting implementation testing of this research.

10. Data and Code Availability

Reference Ansible playbooks, Caddy configurations, Headscale ACLs, and the MCP-proxy systemd unit files used in this deployment are available at `https://github.com/vibhav-aggarwal/lala-net-public` (to be published upon acceptance) under the MIT License.

References

[1] postmarketOS contributors, postmarketos: A real linux distribution for phones, alpine Linux–based mobile operating system built on mainline kernels (2024).
URL `https://postmarketos.org/`

[2] Cloudflare, Inc., Cloudflare tunnel documentation (2024).
URL `https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/`

[3] Oracle Corporation, Oracle cloud infrastructure always free resources (2024).
URL `https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm`

[4] J. Font Brodowski, Headscale contributors, Headscale: An open source, self-hosted implementation of the tailscale control server, software project, BSD-3-Clause license (2024).
URL `https://github.com/juanfont/headscale`

[5] A. Pennarun, How tailscale works, tailscale engineering blog post (2021).
URL `https://tailscale.com/blog/how-tailscale-works`

[6] Nextcloud GmbH, Nextcloud hub documentation (2024).
URL `https://docs.nextcloud.com/`

[7] Home Assistant community, Home assistant: Open source home automation (2024).
URL https://www.home-assistant.io/
[8] J. Mickens, The slow winter, USENIX ;login: 39 (5) (2014).
[9] J. A. Donenfeld, WireGuard: Next generation kernel network tunnel, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2017. doi:10.14722/ndss.2017.23160.
[10] ZeroTier, Inc., Zerotier manual (2024).
URL https://docs.zerotier.com/
[11] Slack Technologies, Nebula: A scalable overlay networking tool with a focus on performance, simplicity, and security (2019).
URL https://github.com/slackhq/nebula
[12] ngrok, Inc., ngrok documentation (2024).
URL https://ngrok.com/docs
[13] fatedier, frp: A fast reverse proxy to expose local servers behind nat or a firewall, software project, Apache-2.0 license (2024).
URL https://github.com/fatedier/frp
[14] Tailscale, Inc., Tailscale funnel: Share a local service on the public internet (2023).
URL https://tailscale.com/kb/1223/funnel
[15] Anthropic, PBC, Introducing the model context protocol (Nov. 2024).
URL https://www.anthropic.com/news/model-context-protocol
[16] Model Context Protocol Working Group, Model context protocol specification (2024).
URL https://modelcontextprotocol.io/specification
[17] S. Sparfenyuk, mcp-proxy contributors, mcp-proxy: A proxy server bridging mcp stdio servers to http/sse clients, v6.4.0, MIT license (2024).
URL https://github.com/sparfenyuk/mcp-proxy
[18] M. Holt, Caddy contributors, Caddy: Fast, multi-platform web server with automatic https (2024).
URL https://caddyserver.com/
[19] OpenWrt contributors, Openwrt project documentation (2024).
URL https://openwrt.org/docs/start
[20] Frappe Technologies, Erpnext: Open-source erp built on the frappe framework (2024).
URL https://github.com/frappe/erpnext
[21] V. Aggarwal, Public cluster topology dashboard, live public dashboard of the deployment: node inventory, 39 CPU cores, 44 GB RAM, 51 GB VRAM, 5 TB storage, $0/month recurring (2026).
URL https://lala.city/servers
[22] U. Gupta, Y. G. Kim, S. Lee, J. Tse, H.-H. S. Lee, G.-Y. Wei, D. Brooks, C.-J. Wu, Chasing carbon: The elusive environmental footprint of computing, in: Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2022, on the dominance of embedded (manufacturing) emissions over operational emissions in modern compute.

Table 1: Topology Overview

Node	Segment	Hardware	OS	Role	Headscale IP
network-server	Cloud	OCI Always-Free VM.Standard.E2.1.Micro	Ubuntu 24.04	Control plane (Headscale, SSH gateway, Caddy) + WireGuard exit hub (7 exits + WARP), Tor gateway, Postfix mail relay, metrics agents (Grafana Alloy, cadvisor, node_exporter), embedded DERP	100.64.0.1
cloud-server	Cloud	OCI Always-Free VM.Standard.E2.1.Micro	Ubuntu 24.04	Fleet uptime/blackbox monitoring host (self-hosted Uptime Kuma, Docker), probing nodes over direct public-IP and Cloudflare-tunnel paths — deliberately independent of the mesh; general-purpose compute	100.64.0.8
office-server	Office	Raspberry Pi 5 (BCM2712)	Kali Linux	Automation & business apps: Home Assistant, Scanopy, ERPNext, ZKTeco ADMS (attendance), Prometheus, MCP; cameras (Frigate/Scrypted/MediaMTX, with a Coral Edge TPU for object-detection inference), biometric access-control (Secureye), and telephony/SMS (voice over Bluetooth-HFP to mobile-server; SMS via its SMSGate API)	100.64.0.2
lab-server	Lab	IBM x3650 (Type 7979, 2× Xeon X5450)	Ubuntu 24.04	Research lab (Neo4j, batch compute)	100.64.0.5
gpu-server	Lab	XtremeMINER X1 frame; ASRock Q270 BTC+, i5-6500, 6× AMD RX 570 4GB	Kali Linux	Media hub + OpenCL (6-GPU)	100.64.0.4
nas-server	Lab	Gigabyte GA-945GCMX-S2, Pentium E2140	Ubuntu 24.04	Data lake (bulk storage)	(LAN-only)
home-server	Home	Raspberry Pi 5 (BCM2712)	Raspberry Pi OS	Personal cloud & home hub: Nextcloud AIO, Vaultwarden, Stalwart mail, Frigate NVR, Wyoming voice, Home Assistant, OpenWA, Syncthing	100.64.0.6
mobile-server	Nomadic	OnePlus 6 (SDM845, 8 GB)	LineageOS 22.2 (Android 15) + Kali chroot	Cellular gateway (SMS via SMSGate, voice via Bluetooth HFP) + always-on edge	100.64.0.7
admin-server	Nomadic	Alienware 17 R2	Windows 10 + Kali (WSL2)	Rendering, training & testing unit (GTX 970M, CUDA); nomadic — travels with the operator and works from any site	100.64.0.3
test-server	Lab	Mac mini (Late 2014; i5-4278U, 8 GB)	Ubuntu 26.04	**Apps/services testing ground**	100.64.0.9
black-router	Nomadic	Netgear R6120	OpenWrt 23.05	WiFi gateway; travels with the nomadic subnet	—

Table 2: Hardware-acquisition strategy: legacy-first, minimal-new

Node	Verified hardware	HW release yr	Rootfs birth	Provenance	Historical cost (INR)	Marginal cost (INR)
home-server	Raspberry Pi 5 (8 GB), 1 TB T7 SSD, Debian 12	2023	2025-05-13	Bought new for the cluster	~13,225	~13,225
office-server	Raspberry Pi 5 (8 GB) build + Google Coral USB Accelerator (Edge TPU)	2023 (Coral 2019)	2025-10-14	Bought new for the cluster	~12,800 + 12,000	~12,800 + 12,000
lab-server	IBM System x3650 (Type 7979), 2× Xeon X5450, 4 GB ECC	2007	2026-02-11	Pre-owned; ex-enterprise decommission	~300,000 (2009 list)	0 (sunk)
admin-server	Alienware 17 R2, i7-4720HQ, GTX 970M, 256 GB SSD	2015	2026-02-23	Secondhand 2021 (Rs 25,000); prior design workstation	~180,000 (2015)	0 (sunk)
gpu-server	BTC-D37 mining board, Celeron 877; up to 12× RX 570 (4 active)	2017	2026-02-27	Operator's own post-mining rig	~550,000 (2020 build)	0 (sunk)
nas-server	Custom build, recycled consumer parts	—	—	Pre-owned, repurposed	not recorded	0
test-server	Mac mini (Late 2014), i5-4278U, 8 GB	2014	2026-10-09	Secondhand, bought for the cluster	~50,000 (2014)	13,500
mobile-server	OnePlus 6 (SDM845, 8 GB), LineageOS + Kali chroot	2018	—	Pre-owned phone, repurposed	~42,598 (2018)	0 (sunk)
black-router	Netgear R6120 AC1200 + OpenWrt 23.05	2016	—	Pre-owned home router	~2,500	0 (sunk)
network-server	OCI Always-Free VM (AMD EPYC slice, 2 vCPU, ~1 GB)	—	2025-10-31	OCI Always-Free tier	0	0
cloud-server	OCI Always-Free VM (same tier)	—	2026-02-28	OCI Always-Free tier	0	0
Total					**≥ Rs 11,63,100 (≈ USD 14,100) — historical**	**≈ Rs 51,525 (≈ USD 625) — marginal**

Table 3: Network access routes: latency, throughput, and path (measured 2026-10-10)

Route (discovery)	Latency	Throughput	Path
LAN, mDNS/Avahi (`.local`)	sub-ms	**1 Gbps** (NIC link negotiated 1000 Mbit/s, confirmed)	single L2 switch, 0 WAN hops
Headscale/Tailscale mesh, **direct**	~13 ms (home-server)	—	peer-to-peer (succeeds only where NAT permits)
Headscale/Tailscale mesh, **DERP-relayed**	37–99 ms	**3.7–13 Mbps** (asymmetric, WAN-upload-limited)	node → embedded DERP on the control-plane VM (Mumbai, ~83 ms) → node
Cloudflare Tunnel (`*.example.com`)	TTFB ~0.5–0.9 s	inbound HTTPS	client → nearest Cloudflare PoP → origin tunnel
SSH gateway (control-plane DNAT)	connect ~0.9 s	—	client → Mumbai VM → target over mesh